INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY, BHUBANESWAR
Department of Computer Science & Engineering | DAA Laboratory (CS3101)

LAB 09 REPORT: GREEDY DESIGN PARADIGM

Student: Rishav Kumar **Roll:** B125099 **Branch:** B.Tech CSE (B1) **Instructor:** Dr. A. K. Dash **Date:** Oct 2026

Index of Questions & Algorithmic Summary

Q. No.	Problem Name	Algorithmic Paradigm	Time	Space	Status
Q1	Fractional Knapsack (Decay)	Deteriorating Resource Greedy	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q2	Canonical Huffman Coding	Min-Heap Prefix Tree & Canonical	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q3	Minimum Refueling Stops	Retroactive Max-Heap (Reverse)	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q4	Minimum Cost Sticks	Min-Heap Huffman Merge Tree	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q5	Candy Distribution	Bi-directional Slope Greedy	$\Theta(n)$	$\Theta(n)$	Verified
Q6	Reorganise String K-Apart	Max-Heap + FIFO Cooldown Queue	$\mathcal{O}(N \log\|\Sigma\|)$	$\mathcal{O}(N + \|\Sigma\|)$	Verified
Q7	Minimise Array Deviation	Two-Way Greedy Monotonic Descent	$\mathcal{O}(n \log(M) \log n)$	$\mathcal{O}(n)$	Verified
Q8	Meeting Rooms Allocation	Interval Coloring / Min-Heap	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Verified
Q9	Hu-Tucker Tree Simulation	3-Phase Compatible Greedy Merge	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	Verified
Q10	Greedy Superstring Open	SCS Heuristic & 2026 AI Disproof	$\mathcal{O}(n^3 L)$	$\mathcal{O}(nL)$	Explored

Q1: Fractional Knapsack with Deterioration Rate

Greedy Exchange Priority · $\mathcal{O}(n \log n)$

Problem Statement: n items with base values v_i, weights w_i, decay rates $\lambda_i > 0$. At time t, value density decays to $\left(\frac{v_i}{w_i}\right) - \lambda_i t$. Knapsack capacity is W. Determine optimal scheduling order and fractional choices to maximize total value.

Exchange Proof & Formulation: Consuming weight w_i' starting at t_s yields $V_i = w_i' \left[\frac{v_i}{w_i} - \lambda_i \left(t_s + \frac{w_i'}{2}\right)\right]$. For two consecutive items A and B, $V(A \to B) - V(B \to A) = w_A w_B (\lambda_A - \lambda_B)$. Since $w_A, w_B > 0$, items with higher decay rates λ_i **must strictly precede** items with lower decay rates to minimize waiting losses. Items are truncated at $t_{\max} = \frac{v_i}{w_i \lambda_i}$ when marginal density reaches zero.

- **Complexity:** Time: $\mathcal{O}(n \log n)$ (dominated by `qsort`), Auxiliary Space: $\mathcal{O}(n)$.
- **Verification Trace:** Items $[(60, 10, 0.5), (100, 20, 0.8), (120, 30, 0.2)]$, $W = 50$: Scheduled Item 2 (31.25%, val 15.625), Item 1 (57.50%, val 8.266), Item 3 (26.67%, val 6.400). Max Total Value = 30.2906.

```
int compareItems(const void *a, const void *b) {
    const Item *ia = (const Item *)a, *ib = (const Item *)b;
    if (ib->lambda > ia->lambda) return 1;
    if (ib->lambda < ia->lambda) return -1;
    return (ib->density > ia->density) ? 1 : -1;
}
```

Q2: Canonical Huffman Coding

Prefix Tree & Canonical Codebook · $\mathcal{O}(n \log n)$

Problem Statement: Given symbol frequencies, construct a prefix-free binary code of minimum expected length, and output the canonical Huffman codebook (where codes are ordered lexicographically by length).

Theoretical Invariant & Canonical Generation: Huffman builds an optimal full binary tree bottom-up by repeatedly merging the two smallest frequency nodes (CLRS Lemma 16.2 & 16.3). Canonical codes standardize transmission by sorting symbols by code length L_i ascending and symbol ASCII: $C_0 = 0, \quad C_i = \left(C_{\{i-1\}} + 1\right) \ll \left(L_i - L_{\{i-1\}}\right)$. Decompressors only require the length array.

- **Complexity:** Time: $\mathcal{O}(n \log n)$ (min-heap + sorting), Auxiliary Space: $\mathcal{O}(n)$.
- **Verification Trace (CLRS 16 Benchmark):** $A:45, B:13, C:12, D:16, E:9, F:5$ ($N = 100$): Canonical codes: $A(L=1):\mathbf{0}, B(L=3):\mathbf{100}, C(L=3):\mathbf{101}, D(L=3):\mathbf{110}, E(L=4):\mathbf{1110}, F(L=4):\mathbf{1111}$. Total bits: 224, Entropy: 2.2199, Expected Length: 2.2400, Coding Efficiency: 99.10%.

```
void buildCanonicalCodes(SymbolCode *codes, int n) {
    qsort(codes, n, sizeof(SymbolCode), compareCanonical);
    unsigned int cur = 0; int curLen = codes[0].length;
    for (int i = 0; i < n; i++) {
        if (codes[i].length > curLen) { cur <<= (codes[i].length - curLen); curLen = codes[i].length; }
        codes[i].canonicalCode = cur++;
    }
}
```

Q3: Minimum Initial Fuel (Reverse Greedy)

Retroactive Max-Heap Simulation · $\mathcal{O}(n \log n)$

Problem Statement: A vehicle needs to reach target D starting with fuel F. Along the way are stations (d_i, f_i) (distance from origin and fuel stock). Find the minimum number of refueling stops required.

Reverse Greedy Strategy: Drive forward as far as possible. Whenever current fuel is insufficient to reach the next station d_i or destination D, retroactively refuel at the passed station offering the **maximum fuel capacity** using a Max-Heap. This guarantees maximal reachable horizon at every step. If heap is empty during a deficit, destination is unreachable (-1).

- **Complexity:** Time: $\mathcal{O}(n \log n)$ (sorting stations + heap ops), Auxiliary Space: $\mathcal{O}(n)$.
- **Verification Trace:** Target $D = 100, F = 10$, Stations $[(10, 60), (20, 30), (30, 30), (60, 40)]$: At $x = 10$: refuels at Station 1 ($+60L \to 60$). At $x = 60$: refuels at Station 4 ($+40L \to 50$). Reaches $D = 100$ with $10L$ fuel. Minimum stops: **2** (Stations 1 and 4).

```
while (currentFuel < distNeeded) {
    if (heap->size == 0) { possible = 0; break; }
    HeapElement best = extractMax(heap);
    currentFuel += best.station.fuel;
    chosenStops[stopsCount++] = best.station;
}
```

Q4: Minimum Cost to Connect Sticks

Huffman Merge Equivalence · $\mathcal{O}(n \log n)$

Problem Statement: Given sticks of lengths $L_1, ..., L_n$, connecting two sticks of lengths x and y costs $x + y$, leaving a stick of length $x + y$. Find the minimum total connection cost to merge all sticks into one.

Mathematical Equivalence to Huffman: Each pairwise merge builds an internal node in a full binary merge tree. Leaf stick L_i contributes $L_i \cdot d_{T(i)}$ to the total cost. Hence, $\text{Total Cost} = \sum_{\{i=1\}}^{n} L_i \cdot d_{T(i)}$, which is mathematically identical to Huffman tree external path length. Greedily extracting the two smallest sticks via a Min-Heap minimizes the sum.

- **Complexity:** Time: $\mathcal{O}(n \log n)$ (Floyd heap build in $\mathcal{O}(n) + n - 1$ extractions), Space: $\mathcal{O}(n)$. 64-bit `long long` prevents overflow.
- **Verification Trace:** Sticks $[1, 8, 3, 5]$: Step 1: Merge $1 + 3 = 4$ (Cost 4). Step 2: Merge $4 + 5 = 9$ (Cost 13). Step 3: Merge $8 + 9 = 17$ (Cost 30). Final stick length: 17, Minimum Total Cost: **30**.

```
while (heap->size > 1) {
    long long a = extractMin(heap), b = extractMin(heap);
    long long cost = a + b; totalCost += cost;
    insertMinHeap(heap, cost);
}
```

Q5: Candy Distribution Problem

Bi-directional Slope Greedy · $\Theta(n)$

Problem Statement: n children stand in line with ratings $r_0, ..., r_{\{n-1\}}$. Each child gets at least 1 candy. Children rated strictly higher than immediate neighbors must receive more candies than them. Minimize total candies.

Bi-directional Decomposition: A single pass fails because descending slopes create right-to-left dependencies.

1. Forward Pass (Left-to-Right): If $r_i > r_{\{i-1\}}$, $c_i = c_{\{i-1\}} + 1$, else $c_i = 1$. (Satisfies left neighbor).
2. Backward Pass (Right-to-Left): If $r_i > r_{\{i+1\}}$, $c_i = \max\left(c_i, c_{\{i+1\}} + 1\right)$. (Satisfies right neighbor while preserving left).

Optimality follows because $c_i = \max\left(c_i^L, c_i^R\right)$ is component-wise minimal.

- **Complexity:** Time: $\Theta(n)$ (two linear scans), Auxiliary Space: $\Theta(n)$ (`ratings` and `candies` arrays).
- **Verification Trace:** Ratings $[1, 0, 2] \rightarrow$ Candies: $\{2, 1, 2\}$. Ratings $[12, 8, 5, 2, 1] \rightarrow$ Candies: $\{5, 4, 3, 2, 1\}$.

```
for (int i = 1; i < n; i++)
    if (ratings[i] > ratings[i - 1]) candies[i] = candies[i - 1] + 1;
for (int i = n - 2; i >= 0; i--)
    if (ratings[i] > ratings[i + 1] && candies[i] < candies[i + 1] + 1)
        candies[i] = candies[i + 1] + 1;
```

Q6: Reorganise String with K-Distance Apart

Max-Heap & Cooldown Queue · $\mathcal{O}(N \log|\Sigma|)$

Problem Statement: Given string S and integer K, rearrange S such that identical characters are at least distance K apart ($j - i \geq K$). If impossible, return an empty string.

Greedy Strategy & Cooldown Barrier: Count character frequencies. At each slot i, greedily place the eligible character with the highest remaining frequency from a Max-Heap (most constrained first). Placed characters enter a circular FIFO cooldown queue of size K with release time $i + K$. If the heap becomes empty while characters remain, pigeonhole deadlock is detected (impossible).

- **Complexity:** Time: $\mathcal{O}(N \log|\Sigma|) = \mathcal{O}(N)$ (since $|\Sigma| \leq 256$), Space: $\mathcal{O}(N + |\Sigma| + K)$.
- **Verification Trace:** $S = \text{aabbcc}, K = 3 \rightarrow$ Result: acbacb (Valid, distance between occurrences is 3). $S = \text{aaabc}, K = 3 \rightarrow$ Frequency of `'a'` is 3, requiring $(3-1) \times 3 + 1 = 7 > 5$ slots $\Rightarrow$ Returns `""` (Impossible).

```
for (int i = 0; i < n; i++) {
    while (!isQueueEmpty(q) && q->array[q->front].releaseTime <= i)
        insertMaxHeap(heap, dequeue(q).item);
    if (heap->size == 0) { possible = 0; break; }
    CharFreq best = extractMax(heap); result[i] = best.ch;
    if (--best.freq > 0) enqueue(q, (CooldownItem){best, i + k});
}
```

Q7: Minimise Deviation in Array

Two-Way Greedy Monotonic Descent · $\mathcal{O}(n \log M \log n)$

Problem Statement: Positive integer array A. Operations: multiply odd by 2, or divide even by 2. Minimize deviation $(\max(A) - \min(A))$.

Monotonic Reduction to One-Way Greedy: Since odd elements can be doubled at most once and evens can only be halved, we pre-multiply every odd element by 2. All elements are now at their upper ceiling! From this state, the **only legal operation is division by 2**. In a Max-Heap, repeatedly pop the maximum M. Update $\Delta = \min(\Delta, M - \text{minVal})$. If M is even, push $\frac{M}{2}$ and update minVal. If M is odd, it cannot be reduced further; search terminates at global optimum.

- **Complexity:** Time: $\mathcal{O}(n \log(M) \log n)$, Auxiliary Space: $\mathcal{O}(n)$ for Max-Heap.
- **Verification Trace:** Input $[4, 1, 5, 20, 3] \rightarrow$ Doubled odds: $[4, 2, 10, 20, 6]$ (Dev 18). Halve $20 \rightarrow 10$ (Dev 8), halve $10 \rightarrow 5$ (Dev 4), halve $6 \rightarrow 3$ (Dev 3). Top is 5 (odd). Search stops. Minimum Deviation = **3**.

```
while (heap->size > 0) {
    int maxV = extractMax(heap);
    if (maxV - minVal < minDev) minDev = maxV - minVal;
    if (maxV % 2 == 0) {
        int half = maxV / 2; if (half < minVal) minVal = half;
        insertMaxHeap(heap, half);
    } else break;
}
```

Q8: Minimum Number of Meeting Rooms

Interval Graph Coloring · $\mathcal{O}(n \log n)$

Problem Statement: Given meeting intervals $[[s_1, e_1], [s_2, e_2], ...]$, find the minimum number of conference rooms required.

Interval Graph Perfection: Conflict graph G is an interval graph, where chromatic number equals clique size: $\chi(G) = \omega(G) = d = \max_t |\{i : s_i \leq t < e_i\}|$. Sort meetings by start time s_i. Maintain a Min-Heap of room end times. If earliest room end time $\leq s_i$, reuse the room; otherwise allocate a new room. Kleinberg & Tardos Theorem 4.4 guarantees optimality.

- **Complexity:** Time: $\mathcal{O}(n \log n)$ (start-time sorting + heap), Auxiliary Space: $\mathcal{O}(n)$.
- **Verification Trace:** Meetings $[[0, 30], [5, 10], [15, 20]]$: Meeting 1 creates Room 1 ($e = 30$). Meeting 2 creates Room 2 ($e = 10$). Meeting 3 arrives at $s = 15 \geq 10$, reuses Room 2 ($e = 20$). Minimum Rooms Needed: **2**.

```
for (int i = 0; i < n; i++) {
    if (heap->size > 0 && heap->array[0].endTime <= meetings[i].start) {
        Room r = extractMinRoom(heap); r.endTime = meetings[i].end;
        insertMinHeap(heap, r); meetings[i].assignedRoom = r.roomId;
    } else {
        Room nr = {++roomCount, meetings[i].end};
        insertMinHeap(heap, nr); meetings[i].assignedRoom = nr.roomId;
    }
}
```

Q9: Hu-Tucker Greedy Simulation (Alphabetic Trees)

3-Phase Alphabetic Tree Merge · $\mathcal{O}(n^2)$

Problem Statement: Given ordered weights $w_1, ..., w_n$, construct an optimal alphabetic binary search tree minimizing $\sum w_i \cdot \text{depth}(i)$ while maintaining strict in-order sequence of leaves.

The Three Hu-Tucker Phases:

1. Combination Phase: In working sequence, merge compatible pair (u, v) with minimum $w(u) + w(v)$. Two nodes are compatible iff no unmerged original leaves exist between them. Replace u with composite node and delete v.
2. Level Assignment Phase: Extract leaf depths d_i from Phase 1 tree (Hu-Tucker theorem guarantees optimal levels).
3. Reconstruction Phase: Bottom-up level-by-level merge pairing adjacent nodes at maximum level. Result preserves planar in-order.

- **Complexity:** Simulation: $\mathcal{O}(n^2)$, reducible to $\mathcal{O}(n \log n)$ with priority queues. Space: $\mathcal{O}(n)$.
- **Verification Trace:** Weights $[10, 2, 3, 1, 5] \rightarrow$ Levels: $d = [1, 3, 4, 4, 2]$. Cost: $10(1) + 2(3) + 3(4) + 1(4) + 5(2) = \mathbf{42}$. Unrestricted Huffman Cost: 41. Alphabetic Preservation Penalty: $+1$ (2.44% overhead for key order).

```
/* Compatible Check: all elements between i and j must be internal nodes */
int compatible = 1;
for (int k = i + 1; k < j; k++)
    if (seq[k]->isLeaf) { compatible = 0; break; }
```

Q10: Greedy Superstring Conjecture (Open Problem)

SCS Overlap & Sept 2026 Disproof · $\mathcal{O}(n^3 L)$

Problem Statement: Given strings $\mathcal{S} = \{s_1, ..., s_n\}$, find the shortest superstring containing every s_i. Explore the 40-year-old 2-approximation greedy conjecture and the recent September 2026 ArXiv disproof paper.

Theoretical Formulation & Modern Disproof:

- **SCS Problem:** NP-hard and APX-hard. Dual to finding the Maximum-Weight Hamiltonian Path in the overlap digraph.
- **Greedy Overlap Heuristic:** Repeatedly merges the pair (u, v) with maximum suffix-to-prefix overlap $\text{ov}(u, v)$.
- **The 40-Year-Old Conjecture:** Hypothesized $|T_{\text{greedy}}| \le 2.0$ (Gallant et al. 1980, Tarhio & Ukkonen 1988).
- **September 2026 Disproof:** An ArXiv paper constructed a counterexample family with even string length $k \ge 10$ yielding: $R(k) \ge \frac{9k+2}{4k+4} \to \frac{9}{4} = 2.25 \text{ as } k \to \infty$. This strictly disproves the 2-approximation conjecture! The complex counterexample was discovered using AI automated reasoning.
- **Verification Trace:** Fragments `{"CATG", "ATGC", "TGCA", "GCAT"}` → Greedy merges with overlap 3 producing `"CATGCAT"` (Length 7, $\text{OPT} = 7$, Ratio 1.000). Pathological trap instances demonstrate greedy overlap lock-in.

```
char *greedySuperstring(int n, char **inputStrings) {
    int count = n; char **active = cloneStrings(inputStrings, n);
    while (count > 1) {
        int bestU = -1, bestV = -1, maxOv = -1;
        for (int i = 0; i < count; i++)
            for (int j = 0; j < count; j++)
                if (i != j && computeOverlap(active[i], active[j]) > maxOv)
                    { maxOv = computeOverlap(active[i], active[j]); bestU = i; bestV = j; }
        active[bestU] = mergeStrings(active[bestU], active[bestV], maxOv);
        deleteString(active, &count, bestV);
    }
    return active[0];
}
```

Algorithmic Synthesis & Concluding Remarks

Q. No.	Problem Name	Key Theoretical Invariant	Core Data Structure	Efficiency
Q1	Fractional Knapsack	Decay exchange priority (λ_i) ordering	Sorted Array + Cutoff	$\mathcal{O}(n \log n)$
Q2	Canonical Huffman	Sibling optimality at maximum depth	Min-Heap + Bitwise Shifts	$\mathcal{O}(n \log n)$
Q3	Refueling Stops	Retroactive maximal fuel replenishment	Max-Heap (Reverse Greedy)	$\mathcal{O}(n \log n)$
Q4	Connect Sticks	Huffman merge tree external path length	Min-Heap (64-bit int)	$\mathcal{O}(n \log n)$
Q5	Candy Distribution	Bi-directional slope constraint decoupling	Two Linear Pass Arrays	$\Theta(n)$
Q6	Reorganise String	Most-constrained character priority + barrier	Max-Heap + FIFO Queue	$\mathcal{O}(N \log \lvert\Sigma\rvert)$
Q7	Array Deviation	Monotonic descent from upper bound ceiling	Max-Heap Priority Queue	$\mathcal{O}(n \log M \log n)$
Q8	Meeting Rooms	Interval chromatic number $\chi(G) = \omega(G) = d$	Min-Heap of End Times	$\mathcal{O}(n \log n)$
Q9	Hu-Tucker Trees	Compatible pair merge preserves optimal depth	Working Sequence List	$\mathcal{O}(n^2)$
Q10	Superstring Open	Max-overlap Hamiltonian path & 2026 disproof	Overlap Digraph / Greedy	$\mathcal{O}(n^3 L)$

Key Pedagogical Takeaways:

1. **Greedy Choice Property & Exchange Arguments:** Proven in Q1 and Q3; exchanging an optimal solution's choice with the greedy choice never degrades the objective value.
2. **Matroids & Independence Systems:** Greedy guarantees exact optimality when the search space forms a matroid (e.g. Huffman codes in Q2/Q4, Interval coloring in Q8).
3. **Priority Queue Dynamics:** Min/Max-heaps efficiently track dynamic bounds in $\mathcal{O}(\log n)$ time, turning quadratic searches into log-linear scans.
4. **Limits of Heuristics (Q10):** Overlap-greedy heuristics can suffer from myopic choices, illustrating why approximation proofs require rigorous bounds and can be challenged by modern AI-assisted disproofs.

STUDENT CERTIFICATION: I hereby certify that all mathematical formulations, greedy exchange proofs, C implementations, and traces in this Lab 09 report represent my verified academic work.

Signature: Rishav Kumar (Roll: B125099, CSE-B1)

FACULTY EVALUATION: Verified algorithmic correctness, greedy exchange arguments, priority queue invariants, and test traces for Lab 09.

Instructor Signature: Dr. Ajaya Kumar Dash